



Asynchronous Programming

Kamal kumar mukiri, Apt Computing Labs

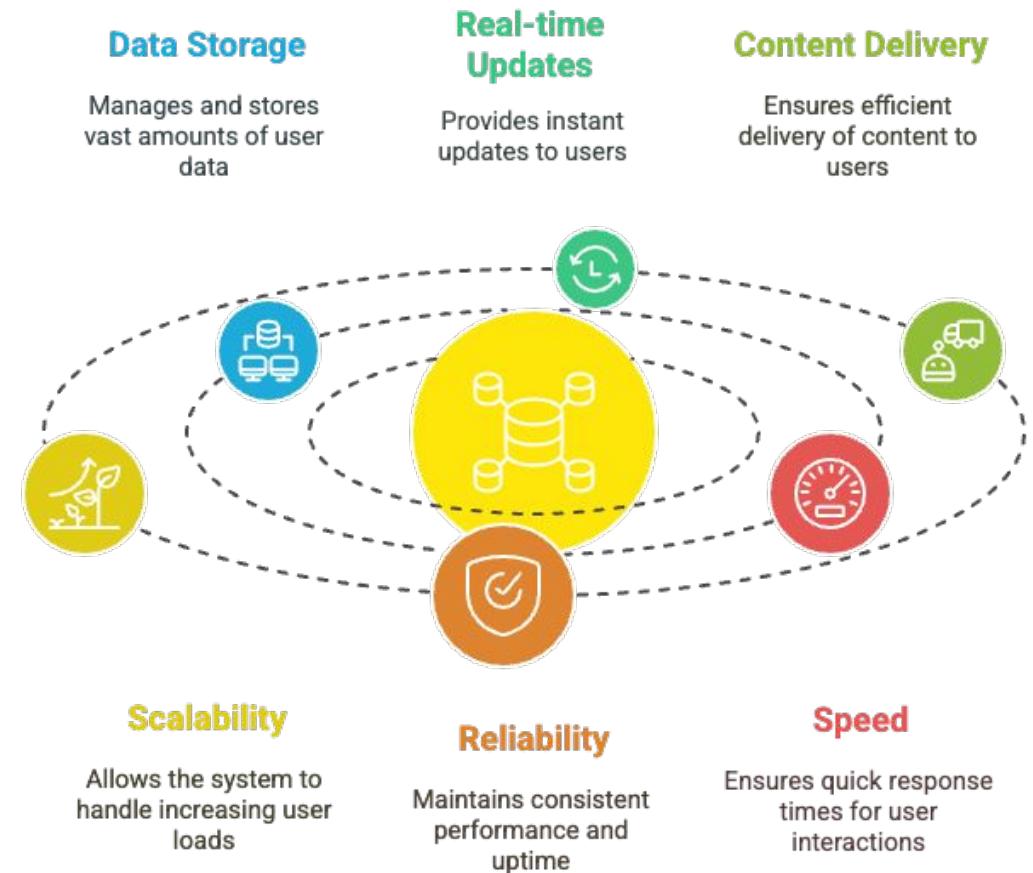
ASYNC PROGRAMMING

- Sync to Async
 - Futures
 - Async Tasks on a Thread Pool
 - Async Designs
- Async Client and Server
 - Send and Receive Packets
- Introduction to “tokio” crate

Problem statement:

"Imagine you're tasked with building the backend for the next big social media platform. Let's call it '**ConverseHub**'. Users will send messages, receive notifications, and engage in real-time chats. From day one, you know this needs to handle *millions* of concurrent connections, process messages instantly, and respond to requests without delay."

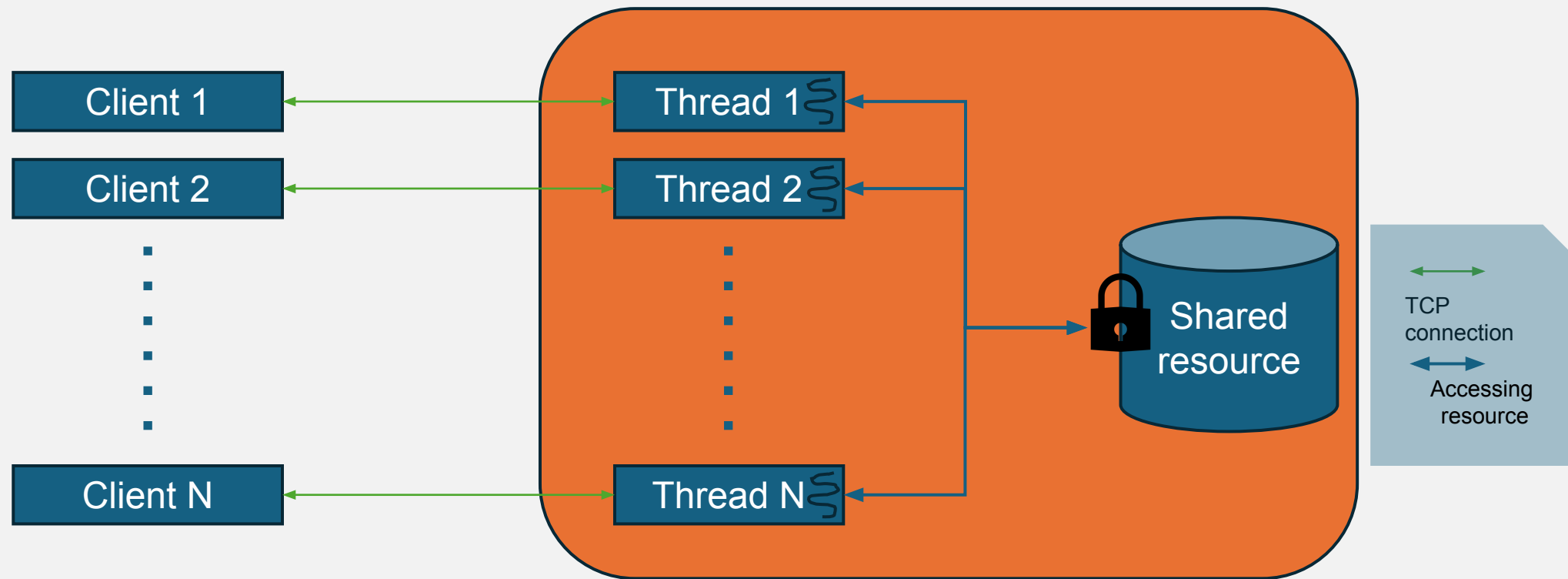
Server Architecture Overview



Designing a Concurrent TCP

Solution 1:

- Create a thread for each connection and handle the messages separately.



Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc).

- Read only

- Mutable

Channels

Atomic Variables

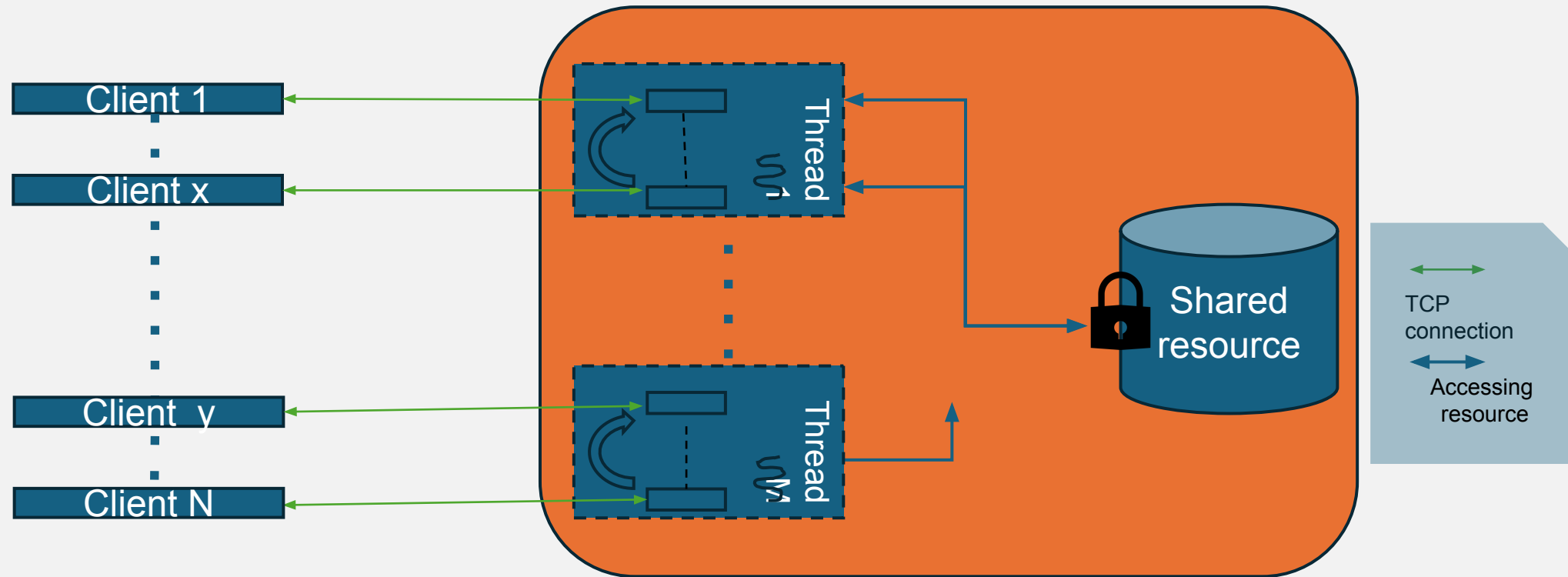


What if the number of connections are huge?

cont...

Solution 2:

- Create thread pool (M number threads) and group the connections (N/M).
- Each thread handles N/M number of connections at max. (using epoll/select)



Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc).

- Read only

- Mutable

Channels

Atomic Variables



What if a connection/reading the message
takes
long time and others are waiting?

Multiple ways to implement async tasks in C/C++:

- Threads (std::thread in C++ / pthread in C)
- std::async (C++11+)
- Coroutines (co_await, co_yield) – C++20
- Event Loops (libevent, libuv, Boost.Asio)
- Thread Pools (std::thread pool, TBB, OpenMP)
- Asynchronous I/O (epoll, select, kqueue, IOCP)
- Signal Handling and Callbacks
- Futures and Promises (std::future, std::promise)

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc).

- Read only

- Mutable

Channels

Atomic Variables

Why Rust is considered more robust

??

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc).

- Read only

- Mutable

Channels

Atomic Variables

- Event Loops (libevent, libuv, Boost.Asio)
- Thread Pools (std::thread pool, TBB, OpenMP)
- Asynchronous I/O (epoll, select, kqueue, IOCP)
- Signal Handling and Callbacks
- Futures and Promises (std::future, std::promise)

Using “std::sync”

```
1  #include <iostream>
2  #include <future>
3
4  using namespace std;
5  int compute() {
6      //print current thread id
7      std::cout << "Thread ID: " << std::this_thread::get_id() << std::endl;
8      return 42;
9  }
10
11 int main() {
12     std::future<int> result = std::async(std::launch::async, compute);
13     std::future<int> result2 = std::async(std::launch::async, compute);
14     std::future<int> result3 = std::async(std::launch::async, compute);
15     std::future<int> result4 = std::async(std::launch::async, compute);
16
17
18     std::cout << "Result: " << result.get() << std::endl; // Blocks until done
19     std::cout << "Result2: " << result2.get() << std::endl; // Blocks until done
20     std::cout << "Result3: " << result3.get() << std::endl; // Blocks until done
21     std::cout << "Result4: " << result4.get() << std::endl; // Blocks until done
22     return 0;
23 }
```

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc).

- Read only

- Mutable

Channels

Atomic Variables

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc).

- Read only

- Mutable

Channels

Atomic Variables

RUST provides “async” tasks...

Few words on "async" programming:



Enables **efficient** and **scalable** applications, especially when handling multiple tasks concurrently.

async - returns a *Future* instead of blocking the current thread

Introduction to threads
Thread operations
- Create/Join
Passing arguments
Collecting return values
Sharing data (using Arc).
- Read only
- **Mutable**
Channels
Atomic Variables

How to dive

??

Synchronous vs Asynchronous:

Concurrency vs Parallelism:

Futures and Promises:

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc).

- Read only

- Mutable

Channels

Atomic Variables

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc).

- Read only

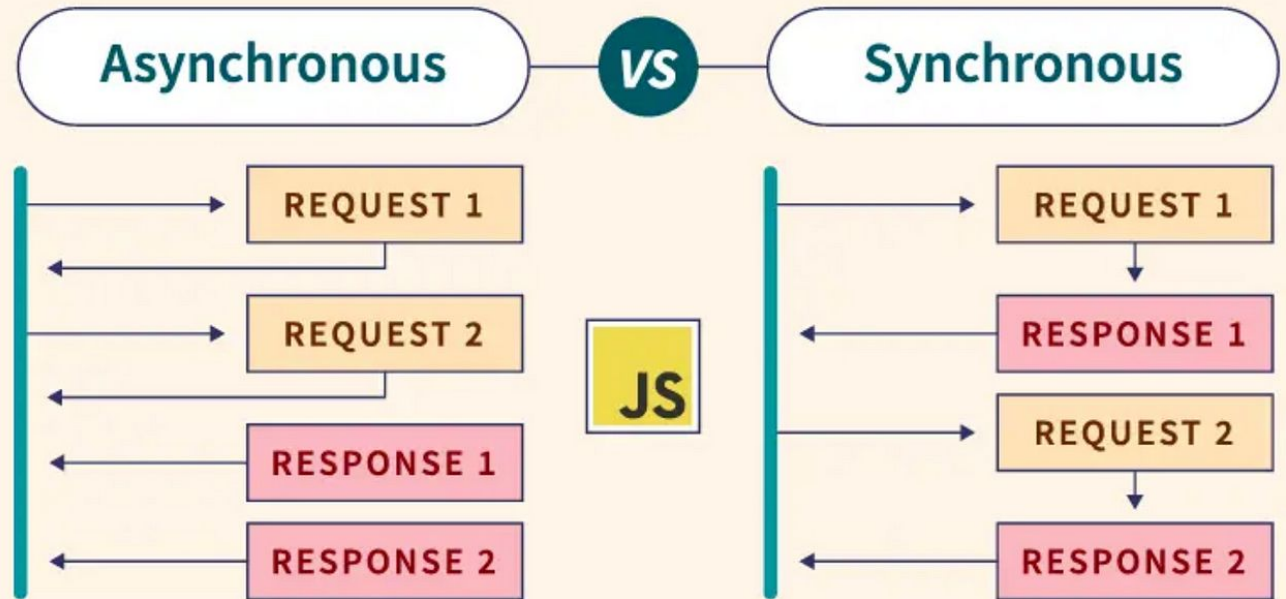
- Mutable

Channels

Atomic Variables

RUST provides “async” tasks...

Clear line between synchronous and asynchronous:



Source:

<https://gurindernarang.medium.com/asynchronous-programming-callbacks-async-await-promise-s-in-js-19e294e84a79>

A Generic flow:

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

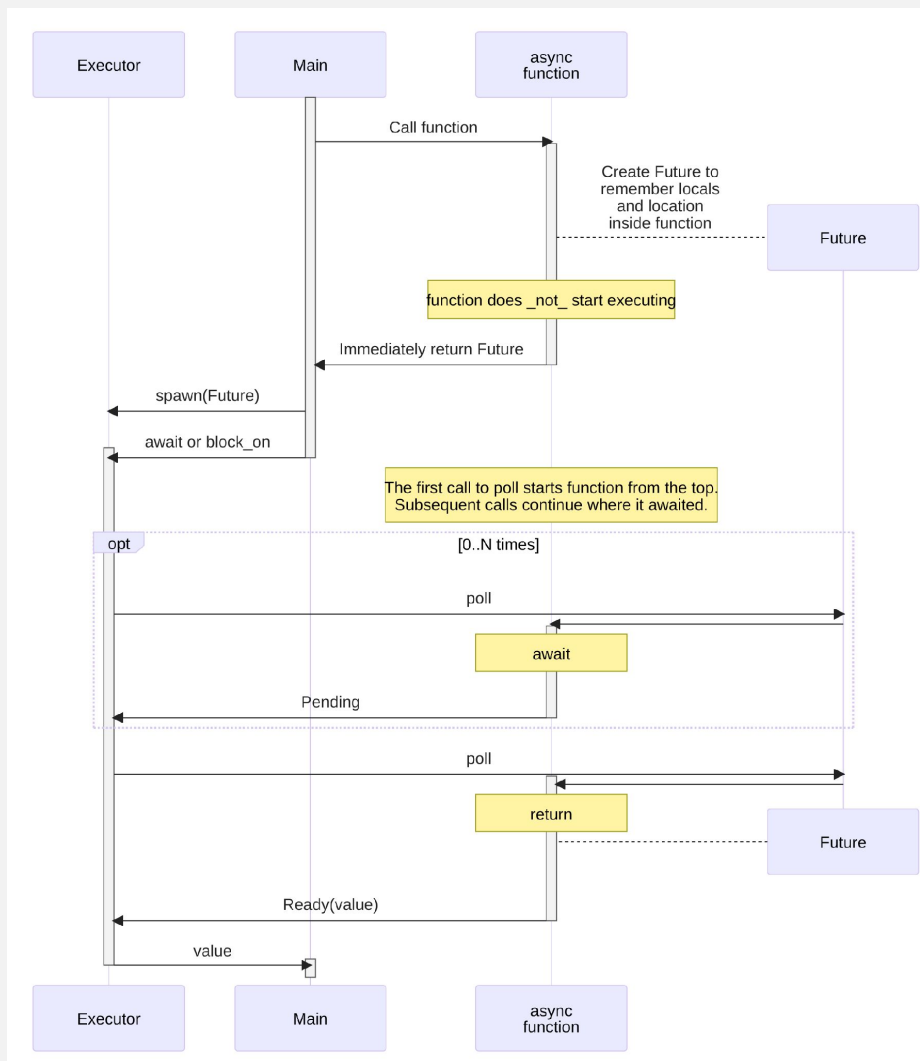
Sharing data (using Arc).

- Read only

- Mutable

Channels

Atomic Variables



Source: Explanation of ARA Applications in Rust AUTOSAR AP R24-11

Beautiful questions:

1. Does Rust run async functions in a separate thread?

- No

- Async functions are executed on an **async runtime** like **Tokio** or **async-std**, which manages tasks cooperatively. Instead of assigning one thread per task, the runtime schedules tasks to execute on a thread pool or single-threaded event loop, switching between tasks as they become ready (e.g., when I/O completes).

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc)

- Read only

- Mutable

Channels

Atomic Variables

Beautiful questions:

2. If it does not run in a separate thread, how do we get the performance?

- Performance in async Rust comes from non-blocking I/O and cooperative multitasking:
- Tasks yield control when they are waiting (e.g., on I/O), allowing other tasks to execute on the same thread.
- By avoiding thread creation and blocking, async programs can handle thousands of concurrent tasks efficiently, making better use of resources.

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc)

- Read only

- Mutable

Channels

Atomic Variables

Implementation of “async” tasks:

1. Asynchronous Tasks by Default

An **asynchronous task** in Rust is represented as a Future. It describes a computation that can be paused and resumed, but it doesn't imply the use of threads. Instead, the runtime decides how to execute these tasks.

- If the runtime uses a **single-threaded executor**, all tasks are run on the same thread.
- If the runtime uses a **multi-threaded executor**, tasks can be executed across multiple threads.

The asynchronous tasks are **cooperative**. They yield control when they're waiting (e.g., on I/O or timers), allowing the runtime to schedule other tasks efficiently without blocking.

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc)

- Read only

- Mutable

Channels

Atomic Variables

Implementation of “async” tasks by “tokio”:

2. Threading with **Tokio** Runtime

Tokio is the most commonly used runtime, and it offers:

a. Multi-threaded Runtime (**Default in `#[tokio::main]`**)

- Tasks can be distributed across multiple threads.
- Useful for CPU-bound and high-concurrency applications.

```
#[tokio::main]
async fn main() {
    let handle1 = tokio::spawn(async {
        println!("Task 1 running on a thread.");
    });

    let handle2 = tokio::spawn(async {
        println!("Task 2 running on a thread.");
    });

    // Wait for tasks to complete
    handle1.await.unwrap();
    handle2.await.unwrap();
}
```

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc)

- Read only

- Mutable

Channels

Atomic Variables

Implementation of “async” tasks by “tokio”:

b. Current-thread Runtime (flavor = "current_thread")

- All tasks run on the same thread.
- Suitable for lightweight applications or where multi-threading is unnecessary.

```
#[tokio::main(flavor = "current_thread")]
async fn main() {
    let handle1 = tokio::spawn(async {
        println!("Task 1 running.");
    });

    let handle2 = tokio::spawn(async {
        println!("Task 2 running.");
    });

    // Wait for tasks to complete
    handle1.await.unwrap();
    handle2.await.unwrap();
}
```

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc).

- Read only

- Mutable

Channels

Atomic Variables

Few points:

Asynchronous != Multithreading: Async tasks are about non-blocking concurrency, not threads. They only consume resources when actively doing work.

Threading is runtime-dependent: Async tasks may use multiple threads if the runtime supports it.

Tasks are cooperative: Async tasks yield control when waiting, enabling efficient scheduling without needing multiple threads.

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc)

- Read only

- Mutable

Channels

Atomic Variables

Implementation of “async” tasks:

1. Asynchronous Tasks by Default

An **asynchronous task** in Rust is represented as a Future. It describes a computation that can be paused and resumed, but it doesn't imply the use of threads. Instead, the runtime decides how to execute these tasks.

- If the runtime uses a **single-threaded executor**, all tasks are run on the same thread.
- If the runtime uses a **multi-threaded executor**, tasks can be executed across multiple threads.

The asynchronous tasks are **cooperative**. They yield control when they're waiting (e.g., on I/O or timers), allowing the runtime to schedule other tasks efficiently without blocking.

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc)

- Read only

- Mutable

Channels

Atomic Variables

Beautiful questions:

1. Does Rust run async functions in a separate thread?

- **No**, Rust's async functions do not inherently run in separate threads.
- Async functions are executed on an **async runtime** like **Tokio** or **async-std**, which manages tasks cooperatively. Instead of assigning one thread per task, the runtime schedules tasks to execute on a thread pool or single-threaded event loop, switching between tasks as they become ready (e.g., when I/O completes).

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc)

- Read only

- Mutable

Channels

Atomic Variables

Beautiful examples:

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc)

- Read only

- Mutable

Channels

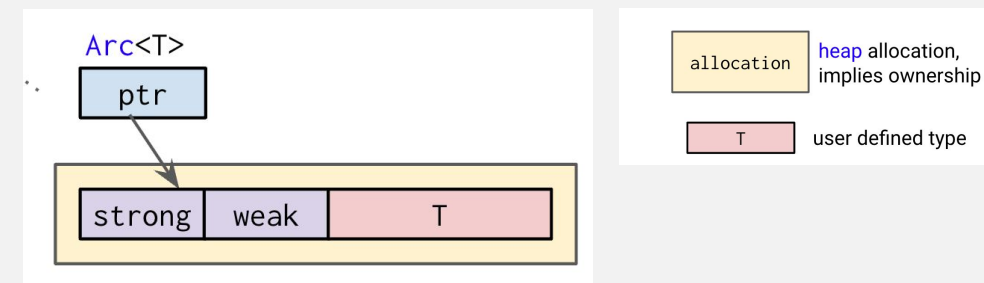
Atomic Variables

```
1 use tokio;
2 use std::thread;
3 use std::time::Duration;
4
5 async fn fun_a() {
6     thread::sleep(Duration::from_secs(2));
7     println!("fun_a is done!!!!");
8 }
9
10 async fn fun_b() {
11     thread::sleep(Duration::from_secs(2));
12     println!("fun_b is done!!!!");
13 }
14 async fn fun_c() {
15     thread::sleep(Duration::from_secs(2));
16     println!("fun_c is done!!!!");
17 }
18 async fn fun_d() {
19     thread::sleep(Duration::from_secs(2));
20     println!("fun_d is done!!!!");
21 }
22 async fn fun1() {
23     println!("Calling fun_a");
24     fun_a().await;
25     println!("Calling fun_b");
26     fun_b().await;
27     println!("Calling fun_c");
28     fun_c().await;
29     println!("Calling fun_d");
30     fun_d().await;
31 }
32
33 #[tokio::main]
34 async fn main() {
35     println!("Beginning!!!");
36     println!("Calling fun1");
37     let fut1 = fun1();
38     println!("Waiting for fut1!!!");
39     fut1.await;
40     println!("Waiting for fut1 is done!!!!");
41 }
```

- 1: Importing “tokio” module.
- 22: “fun1” yields at every line where it is waiting for other “async” functions.

Writing a simple “async” function:

```
1 use std::sync::Arc; ①
2 use std::thread;
3
4 fn main() {
5     // Create some read-only data (a vector of integers)
6     let data = vec![1, 2, 3, 4, 5];
7
8     ② // Wrap the data in Arc to allow shared ownership among threads
9     let shared_data = Arc::new(data);
10
11     // Create a vector to hold the thread handles
12     let mut handles = vec![];
13
14     for i in 0..5 {
15         ③ // Clone the Arc to share the data with each thread
16         let data_clone = Arc::clone(&shared_data);
17
18         // Spawn a new thread
19         let handle = thread::spawn(move || {
20             // Each thread can read the shared data
21             println!("Thread {}: {:?}", i, data_clone); ③
22         });
23
24         // Store the thread handle
25         handles.push(handle);
26     }
27
28     // Wait for all threads to finish
29     for handle in handles {
30         handle.join().unwrap();
31     }
32 }
```

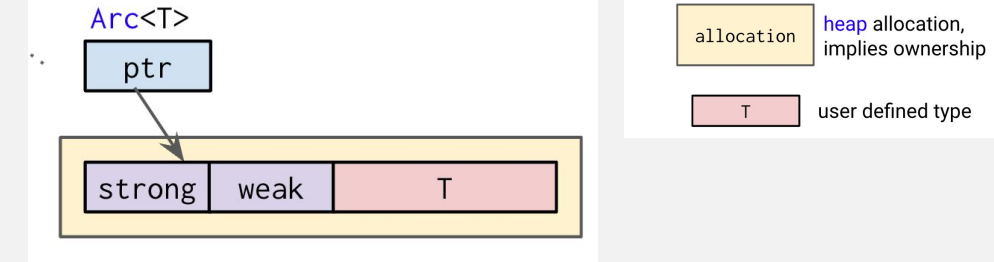


1. Using “std::sync::Arc;
2. Wrapping “data” with “Arc”, protecting with “Arc”
3. Cloning the container “Arc” and passing to thread
4. Thread accesses same heap location

Note: In Rust, when you wrap data in **Arc**, the data itself is moved onto the **heap**, and Arc provides a reference to that data.

Sharing data (immutable): Using “Arc”

```
1 use std::sync::Arc;
2 use std::thread;
3
4 fn main() {
5     // Create some read-only data (a vector of integers)
6     let data = vec![1, 2, 3, 4, 5];
7
8     // Wrap the data in Arc to allow shared ownership among threads
9     let shared_data = Arc::new(data);
10
11     // Create a vector to hold the thread handles
12     let mut handles = vec![];
13
14     for i in 0..5 {
15         // Clone the Arc to share the data with each thread
16         let data_clone = Arc::clone(&shared_data);
17
18         // Spawn a new thread
19         let handle = thread::spawn(move || {
20             // Each thread can read the shared data
21             println!("Thread {}: {:?}", i, data_clone);
22         });
23
24         // Store the thread handle
25         handles.push(handle);
26     }
27
28     // Wait for all threads to finish
29     for handle in handles {
30         handle.join().unwrap();
31     }
32 }
```



1. Using “std::sync::Arc;”
2. Wrapping “data” with “Arc”, protecting with “Arc”
3. Cloning the container “Arc” and passing to thread
4. Thread accesses same heap location

Note: In Rust, when you wrap data in **Arc**, the data itself is moved onto the **heap**, and Arc provides a reference to that data.

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc).

- Read only

- Mutable

Channels

Atomic Variables

RUST provides “async” tasks...

Introduction to File System handling

Ref: <https://doc.rust-lang.org/std/fs/index.html>

- **Reading Files:** `File::open()`, `read_to_string()`, `BufReader`.
- **Writing Files:** `File::create()`, `write_all()`.
- **Appending to Files:** `OpenOptions::new().append(true)`.
- **Binary Files:** `read_to_end()` for reading binary data.
- **File Deletion:** `fs::remove_file()`.

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc).

- Read only

- Mutable

Channels

Atomic Variables

Introduction to socket handling

TCP Socket:

- `TcpListener::bind("127.0.0.1:7878")`: Binds the server to the specified IP address and port.
- `listener.incoming()`: Accepts incoming client connections.
- `stream.read(&mut buffer)`: Reads data from the connected client.
- `stream.write_all(&buffer[0..n])`: Sends data back to the client.
- `TcpStream::connect("127.0.0.1:7878")`: Connects the client to the server.

UDP Socket:

- `UdpSocket::bind("127.0.0.1:8080")`: Binds the UDP socket to a specific address and port.
- `socket.recv_from(&mut buffer)`: Receives a datagram from a client.
- `socket.send_to(b"Hello from server", src)`: Sends a response back to the client.

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc).

- Read only

- Mutable

Channels

Atomic Variables

Read/Write “json” files

- Add Dependencies in Cargo.toml

```
[dependencies]
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
```

- Writing to a JSON file

```
// Define the structure that represents your data
#[derive(Serialize, Debug)]
struct Course {
    name: String,
    year: u16,
    topics: Vec<String>,
}
```

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc).

- Read only

- Mutable

Channels

Atomic Variables

Read/Write “json” files

<https://crates.io/crates/serde>

- Add Dependencies in Cargo.toml

```
[dependencies]
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
```

- Writing to a JSON file

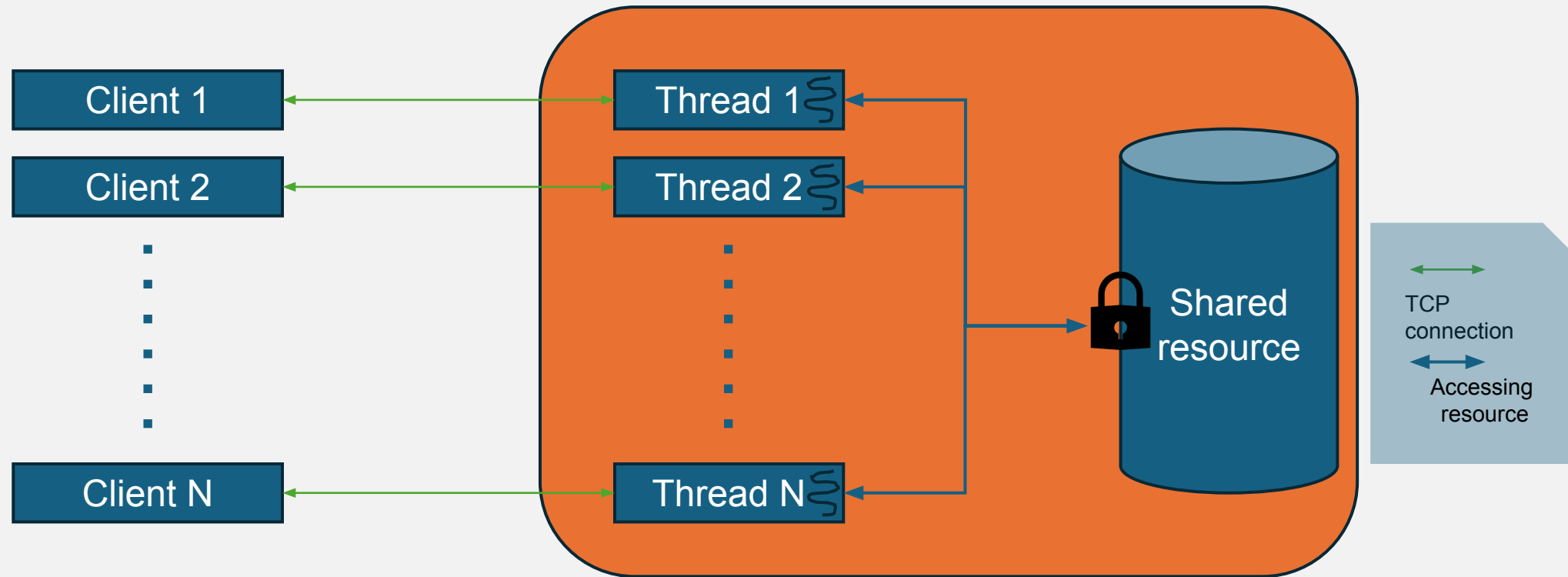
```
// Define the structure that represents your data
#[derive(Serialize, Debug)]
struct Course {
    name: String,
    year: u16,
    topics: Vec<String>,
}
```

Introduction to threads
Thread operations
- Create/Join
Passing arguments
Collecting return values
Sharing data (using Arc).
- Read only
- Mutable
Channels
Atomic Variables

Designing a Concurrent TCP

Solution 1:

- Create a thread for each connection and handle the messages separately.
- **Drawback: Consuming huge memory?** (in case of 1,000,000 connections)



Thread Operations: Create and Join

```
1 1 use std::thread;
2
3 2 fn main() {
4 2 let t1 = thread::spawn(||{
5     println!("Thread created !!!");
6     });
7
8     println!("Thread joined : {:?}" , t1);
9
10    t1.join();
11 }
```

1. Using thread modules
2. **Creating** thread and passing closure (function) which prints “Thread created!!!”.
3. **Joining** the thread in main thread.

Passing variables:

```
1 use std::thread;
2 1 use std::env;
3
4 ✓ fn main() {
5
6     let mut tids = vec![];
7     2 for arg in env::args().skip(1) {
8     let tid = thread::spawn(move || {
9         println!("Thread created !!!");
10        println!("Argumen= {:?}", arg); 3
11    });
12    tids.push(tid);
13
14
15    for tid in tids {
16        tid.join();
17    }
18 }
```

1. Using “env” module
2. Collecting arguments (values) using “iterator” and moving to thread newly created
3. Accessing moved values

Returning value from thread:

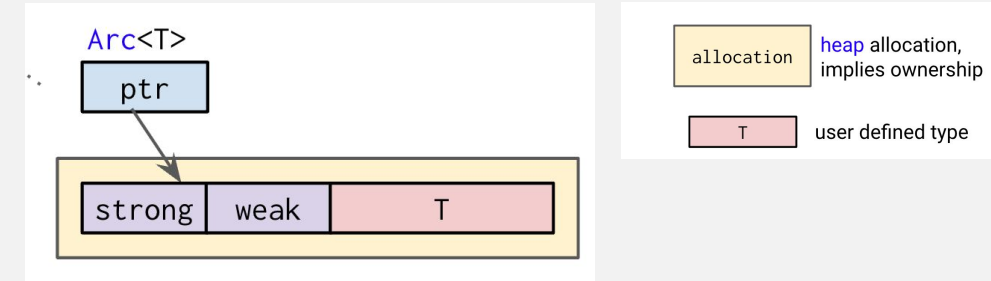
```
1 use std::thread;
2 use std::env;
3
4
5 fn thread_handler(arg: String) -> Result<i32, &'static str> {
6
7     println!("Thread created !!!");
8     println!("Argumen= {:?}",
9         getting handled in thread id {:?}", arg, thread::current().id());
10    Ok(100)
11 }
12
13 fn main() {
14     let mut tids = vec![];
15     for arg in env::args().skip(1) {
16         1 let tid: thread::JoinHandle<Result<i32, &str>> = thread::spawn(move || {
17             thread_handler(arg)
18         });
19         tids.push(tid);
20     }
21
22     for tid in tids {
23         1 match tid.join() {
24             Ok(_) => println!("Executed properly"),
25             Err(msg) => println!("Error value from thread {:?}", msg)
26         }
27     }
28 }
29
```

1. Define the return type of “closure” which is getting executed in thread.
2. Returning value
3. Handling return value of each thread

Sharing data (immutable): Using “Arc”

```
1 use std::sync::Arc; 1
2 use std::thread;
3
4 fn main() {
5     // Create some read-only data (a vector of integers)
6     let data = vec![1, 2, 3, 4, 5];
7
8     2 // Wrap the data in Arc to allow shared ownership among threads
9     let shared_data = Arc::new(data);
10
11     // Create a vector to hold the thread handles
12     let mut handles = vec![];
13
14     for i in 0..5 {
15         3 // Clone the Arc to share the data with each thread
16         let data_clone = Arc::clone(&shared_data);
17
18         // Spawn a new thread
19         let handle = thread::spawn(move || {
20             // Each thread can read the shared data
21             println!("Thread {}: {:?}", i, data_clone); 3
22         });
23
24         // Store the thread handle
25         handles.push(handle);
26     }
27
28     // Wait for all threads to finish
29     for handle in handles {
30         handle.join().unwrap();
31     }
32 }
```

Note: In Rust, when you wrap data in **Arc**, the data itself is moved onto the **heap**, and Arc provides a reference to that data.



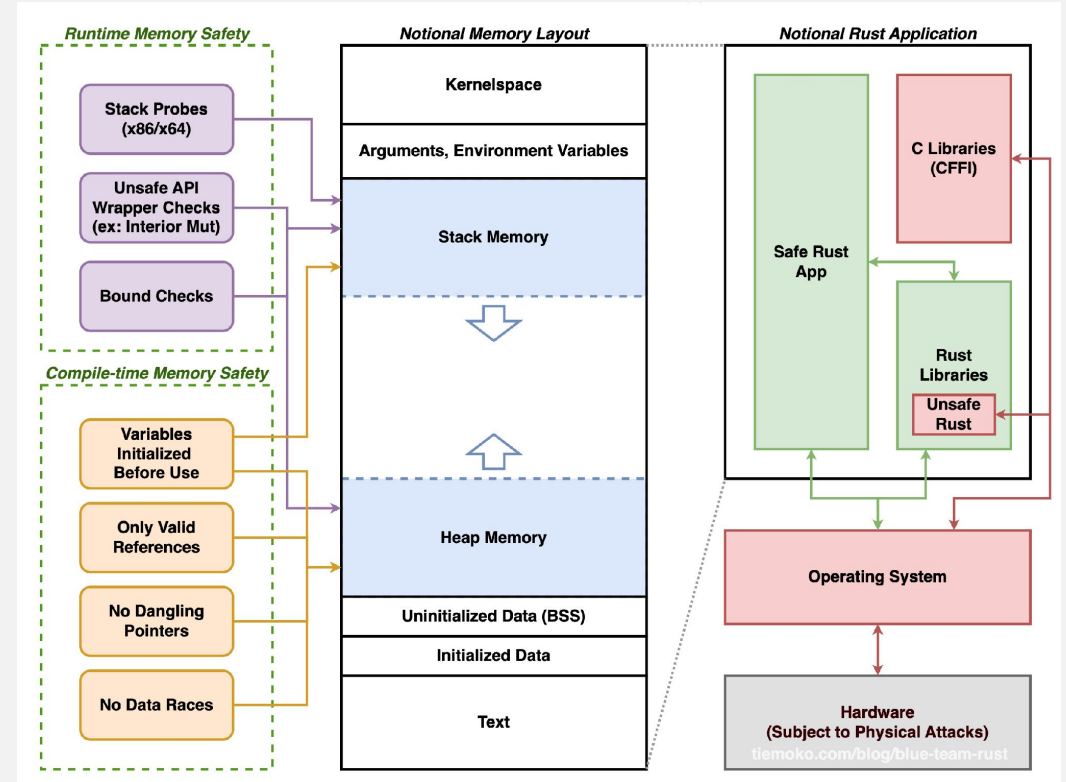
1. Using “std::sync::Arc;”
2. Wrapping “data” with “Arc”, protecting with “Arc”
3. Cloning the container “Arc” and passing to thread
4. Thread accesses same heap location

Sharing data (immutable): Using “Arc”

- Rust compilation fails if shared data is being modified.
- Let us recall one of the principles of Rust Programming

Principles of Rust Programming

- Costs should be apparent to the Programmer
- Basic operations should remain simple.
- Potentially expensive operations should be explicit.



Fearless concurrency:

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc).

- Read only

- Mutable

Channels

Atomic Variables

- Handling concurrent programming safely and efficiently is another of Rust's major goals.
- Concurrent programming, where different parts of a program execute independently, and parallel programming, where different parts of a program execute at the same time, are becoming increasingly important as more computers take advantage of their multiple processors.
- Historically, programming in these contexts has been difficult and error prone: Rust hopes to change that.

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc).

- Read only

- **Mutable**

Channels

Atomic Variables

What if programmer needs to edit the data ???

Using “Arc” + “Mutex” to edit the

data:

```
1 use std::sync::{Arc, Mutex};
2 use std::thread;
3
4 fn main() {
5     // Create some mutable data (a vector of integers)
6     let data = vec![1, 2, 3, 4, 5];
7
8     // Wrap the data in Arc and Mutex to allow shared mutable ownership among threads
9     let shared_data = Arc::new(Mutex::new(data));
10
11    // Create a vector to hold the thread handles
12    let mut handles = vec![];
13
14    for i in 0..5 {
15        // Clone the Arc to get a new reference for each thread
16        let data_clone = Arc::clone(&shared_data);
17
18        // Spawn a new thread
19        let handle = thread::spawn(move || {
20            // Lock the mutex to access the shared data
21            let mut data = data_clone.lock().unwrap();
22            data.push(i); // Modify the data
23            println!("Thread {}: {:?}", i, *data);
24        });
25
26        // Store the thread handle
27        handles.push(handle);
28    }
29
30    // Wait for all threads to complete
31    for handle in handles {
32        handle.join().unwrap();
33    }
34
35    // Final state of shared data
36    let final_data = shared_data.lock().unwrap();
37    println!("Final data: {:?}", *final_data);
38 }
```

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc).

- Read only

- Mutable

Channels

Atomic Variables

Using “Arc” + “Mutex” to edit the

data:

```
1 use std::sync::{Arc, Mutex};
2 use std::thread;
3
4 fn main() {
5     // Create some mutable data (a vector of integers)
6     let data = vec![1, 2, 3, 4, 5];
7
8     // Wrap the data in Arc and Mutex to allow shared mutable ownership among threads
9     let shared_data = Arc::new(Mutex::new(data));
10
11     // Create a vector to hold the thread handles
12     let mut handles = vec![];
13
14     for i in 0..5 {
15         // Clone the Arc to get a new reference for each thread
16         let data_clone = Arc::clone(&shared_data);
17
18         // Spawn a new thread
19         let handle = thread::spawn(move || {
20             // Lock the mutex to access the shared data
21             let mut data = data_clone.lock().unwrap();
22             data.push(i); // Modify the data
23             println!("Thread {}: {:?}", i, *data);
24         });
25
26         // Store the thread handle
27         handles.push(handle);
28     }
29
30     // Wait for all threads to complete
31     for handle in handles {
32         handle.join().unwrap();
33     }
34
35     // Final state of shared data
36     let final_data = shared_data.lock().unwrap();
37     println!("Final data: {:?}", *final_data);
38 }
```

Introduction to threads

Thread operations

- Create/Join

Passing arguments

Collecting return values

Sharing data (using Arc).

- Read only

- Mutable

Channels

Atomic Variables

Misc..

```
#[derive(Debug)]
1 implementation
struct Color(u8, u8, u8);
impl Color {
    1 fn new(r: u8, g: u8, b: u8) -> Color {
        |   Color(r, g, b)
        |   }

    2 fn print_color (&self) {
        |   println!("Color values: {}, {}, {}", self.0, self.1, self.2);
        |   }
    }

    3 fn main() {
        |   let black: Color = Color(0, 0, 0);
        |   let mut white: Color = Color(255, 255, 255);

        |   println!("Black color values: {}", black.0);
        |   white.0 = 0;
        |   white.print_color();
        |   }
    }
```

Outp

```
ut ./tuple_struct_basic
Black color values: 0
Color values: 0, 255, 255
```

1. Defining structure

- No need of fields names, only types are enough.

2. Creating instance

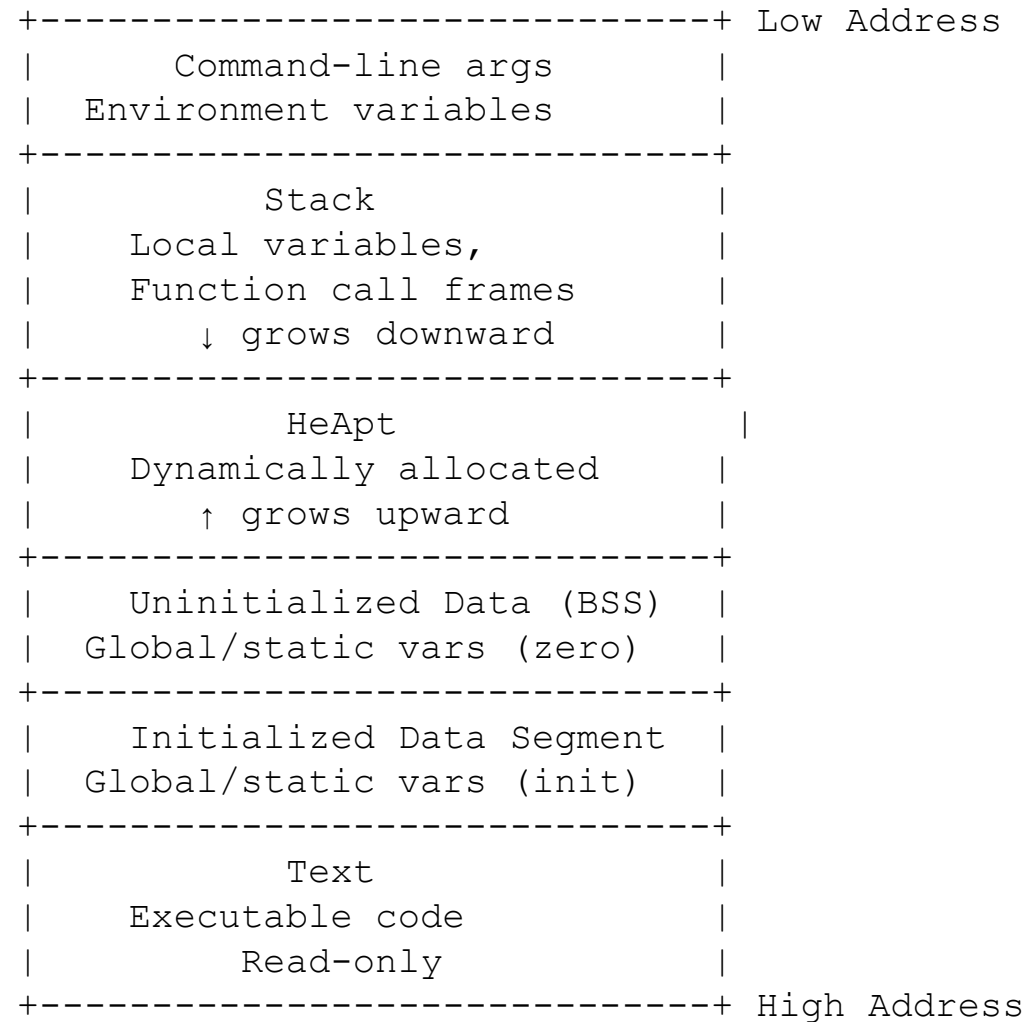
1. Order matters

4. Modifying and accessing members

- Use **.index** to access or modify

1. Process memory
2. Scheduling tasks with Shared CPU
3. Context Switching
4. Creating/Joining Thread
5. Mutex Lock, RW Lock
6. CPU Cache (hit/miss)

1. Process memory



1. Process memory
2. Scheduling tasks with Shared CPU
3. Context Switching